

MongoDB Aggregation Operations — Complete Advanced Guide

Table of Contents

1. [What is Aggregation?](#)
2. [Comparison Operators \(*gt*, *lt*, *gte*, *lte*, *eq*, *ne*\)](#)
3. [The Aggregation Pipeline](#)
4. [Core Pipeline Stages](#)
 - [\\$match](#)
 - [\\$group](#)
 - [\\$project](#)
 - [\\$sort](#)
 - [ParseError: KaTeX parse error: Expected 'EOF', got '&' at position 7: limit &skip](#)
 - [\\$unwind](#)
 - [\\$lookup \(Joins\)](#)
 - [\\$addFields](#)
 - [\\$replaceRoot](#)
 - [\\$facet](#)
 - [ParseError: KaTeX parse error: Expected 'EOF', got '&' at position 8: bucket &bucketAuto](#)
 - [\\$count](#)
5. [Accumulator Operators](#)
6. [Array Operators in Aggregation](#)
7. [String Operators](#)
8. [Date Operators](#)
9. [Conditional Operators \(*cond*, *switch*\)](#)
10. [expr—Using Aggregation Expressions in match](#)
11. [Window Functions \(\\$setWindowFields\)](#)
12. [ParseError: KaTeX parse error: Expected 'EOF', got '&' at position 7: merge &out \(Write Results\)](#)
13. [Aggregation Indexes & Performance](#)
14. [Real-World Practical Examples](#)

1. What is Aggregation?

Aggregation processes **multiple documents** and returns **computed results**. Think of it as SQL's `GROUP BY` , `JOIN` , `HAVING` , and `SELECT` — all combined.

```
// Basic syntax
db.collection.aggregate([
  { stage1 },
  { stage2 },
  { stage3 },
  ...
])
```

Each stage **transforms** the documents and passes them to the next stage like a **conveyor belt**.

2. Comparison Operators

These work inside `$match` , `$expr` , `$cond` , and other stages.

Operator	Meaning	SQL Equivalent
<code>\$gt</code>	Greater than	<code>></code>
<code>\$gte</code>	Greater than or equal	<code>>=</code>
<code>\$lt</code>	Less than	<code><</code>
<code>\$lte</code>	Less than or equal	<code><=</code>
<code>\$eq</code>	Equal	<code>=</code>
<code>\$ne</code>	Not equal	<code>!=</code>
<code>\$in</code>	In array	<code>IN (...)</code>
<code>\$nin</code>	Not in array	<code>NOT IN (...)</code>

✓ Basic Usage in \$match

```
// Find products with price > 100
db.products.aggregate([
  { $match: { price: { $gt: 100 } } }
])

// Find users aged between 18 and 65 (inclusive)
db.users.aggregate([
  { $match: { age: { $gte: 18, $lte: 65 } } }
])

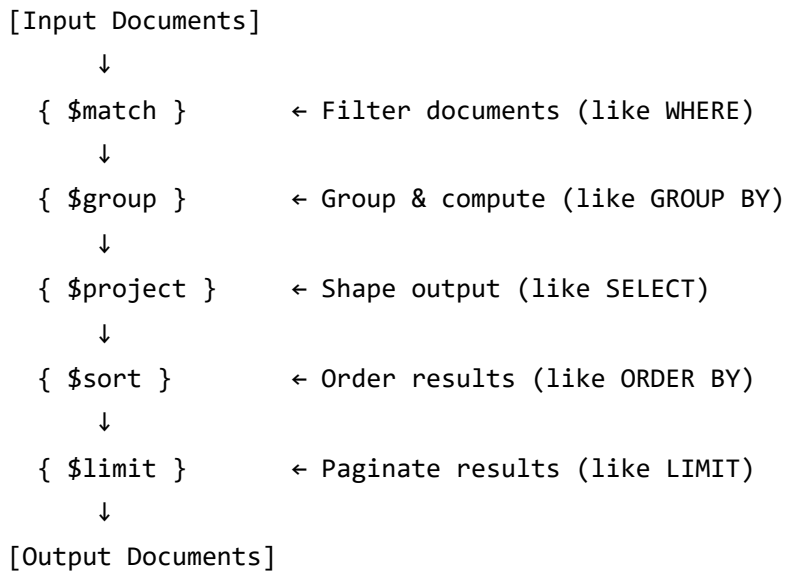
// Find orders NOT equal to "cancelled"
db.orders.aggregate([
  { $match: { status: { $ne: "cancelled" } } }
])

// Find items with quantity in a specific set
db.inventory.aggregate([
  { $match: { quantity: { $in: [10, 20, 30] } } }
])
```

✓ Combining Multiple Conditions

```
// price > 50 AND stock < 100
db.products.aggregate([
  {
    $match: {
      price: { $gt: 50 },
      stock: { $lt: 100 }
    }
  }
])
```

3. The Aggregation Pipeline



4. Core Pipeline Stages

\$match

Filters documents — **always place \$match early** to reduce data flowing downstream.

```
// Find all orders from 2024 with amount > 500
db.orders.aggregate([
  {
    $match: {
      year: 2024,
      amount: { $gt: 500 }
    }
  }
])
```

\$group

Groups documents by a key and applies accumulator functions.

```
// Total sales per category
db.sales.aggregate([
  {
    $group: {
      _id: "$category",          // group by field
      totalRevenue: { $sum: "$amount" },
      avgPrice: { $avg: "$price" },
      maxSale: { $max: "$amount" },
      minSale: { $min: "$amount" },
      count: { $sum: 1 }
    }
  }
])
```

```
// Group by multiple fields
db.orders.aggregate([
  {
    $group: {
      _id: { year: "$year", month: "$month" },
      totalOrders: { $sum: 1 },
      totalRevenue: { $sum: "$amount" }
    }
  }
])
```

```
// Grand total (group everything)
db.sales.aggregate([
  {
    $group: {
      _id: null,
      grandTotal: { $sum: "$amount" }
    }
  }
])
```

\$project

Shapes the output — include, exclude, rename, or compute new fields.

```
db.users.aggregate([
  {
    $project: {
      _id: 0,           // exclude _id
      fullName: 1,      // include field
      email: 1,
      age: 1,
      // Computed field
      discountedPrice: { $multiply: ["$price", 0.9] },
      // Rename a field
      userName: "$name"
    }
  }
])
```

\$sort

```
// Sort by revenue descending, then name ascending
db.sales.aggregate([
  { $sort: { totalRevenue: -1, name: 1 } }
])
```

ParseError: KaTeX parse error: Expected 'EOF', got '&' at position 7:
limit &skip

```
// Pagination: page 2 with 10 items per page
db.products.aggregate([
  { $sort: { price: -1 } },
  { $skip: 10 },    // skip first 10 (page 1)
  { $limit: 10 }    // take next 10 (page 2)
])
```

\$unwind

Deconstructs an **array field** — creates one document per array element.

```
// Sample doc: { name: "Alice", hobbies: ["reading", "coding", "gaming"] }
```

```
db.users.aggregate([
  { $unwind: "$hobbies" }
])
```

```
// Output:
// { name: "Alice", hobbies: "reading" }
// { name: "Alice", hobbies: "coding" }
// { name: "Alice", hobbies: "gaming" }
```

```
// Preserve documents with empty/null arrays
```

```
db.users.aggregate([
  {
    $unwind: {
      path: "$hobbies",
      preserveNullAndEmptyArrays: true
    }
  }
])
```

\$lookup

Performs a **LEFT OUTER JOIN** with another collection.

```

// Join orders with users
db.orders.aggregate([
  {
    $lookup: {
      from: "users",           // target collection
      localField: "userId",    // field in orders
      foreignField: "_id",     // field in users
      as: "userDetails"       // output array name
    }
  }
])

// Advanced $lookup with pipeline (sub-query)
db.orders.aggregate([
  {
    $lookup: {
      from: "products",
      let: { orderId: "$_id", minQty: "$minQuantity" },
      pipeline: [
        {
          $match: {
            $expr: {
              $and: [
                { $eq: ["$orderId", "$$orderId"] },
                { $gte: ["$quantity", "$$minQty"] }
              ]
            }
          }
        },
        { $project: { name: 1, price: 1 } }
      ],
      as: "eligibleProducts"
    }
  }
])

```

\$addFields

Adds new fields **without removing** existing ones (unlike \$project).


```

db.orders.aggregate([
  {
    $addFields: {
      totalWithTax: { $multiply: ["$amount", 1.18] },
      isHighValue: { $gt: ["$amount", 1000] }
    }
  }
])

```

\$replaceRoot

Replaces the root document with a sub-document or a new expression.

```

// Promote nested object to root
db.users.aggregate([
  { $replaceRoot: { newRoot: "$address" } }
])

```

```

// Merge root with new fields
db.users.aggregate([
  {
    $replaceRoot: {
      newRoot: {
        $mergeObjects: ["$$ROOT", { source: "mongodb" }]
      }
    }
  }
])

```

\$facet

Runs **multiple sub-pipelines** in parallel on the same input — ideal for **search result pages**.

```

db.products.aggregate([
  {
    $facet: {
      // Sub-pipeline 1: Category counts
      "categoryCounts": [
        { $group: { _id: "$category", count: { $sum: 1 } } }
      ],
      // Sub-pipeline 2: Price ranges
      "priceStats": [
        {
          $group: {
            _id: null,
            minPrice: { $min: "$price" },
            maxPrice: { $max: "$price" },
            avgPrice: { $avg: "$price" }
          }
        }
      ],
      // Sub-pipeline 3: Top 5 products
      "topProducts": [
        { $sort: { rating: -1 } },
        { $limit: 5 },
        { $project: { name: 1, rating: 1 } }
      ]
    }
  }
])

```

ParseError: KaTeX parse error: Expected 'EOF', got '&' at position 8: bucket &bucketAuto

Categorizes documents into **user-defined ranges (buckets)**.

```
// Manual buckets
db.products.aggregate([
  {
    $bucket: {
      groupBy: "$price",
      boundaries: [0, 50, 100, 500, 1000], // range edges
      default: "Other", // catch-all
      output: {
        count: { $sum: 1 },
        products: { $push: "$name" }
      }
    }
  }
])
```

```
// Automatic equal buckets
db.products.aggregate([
  {
    $bucketAuto: {
      groupBy: "$price",
      buckets: 5, // number of buckets
      output: {
        count: { $sum: 1 },
        avgPrice: { $avg: "$price" }
      }
    }
  }
])
```

\$count

Counts documents passing through the pipeline.

```
db.orders.aggregate([
  { $match: { status: "completed" } },
  { $count: "completedOrders" }
])
// Output: { "completedOrders": 142 }
```

5. Accumulator Operators

Used inside `$group` and `$setWindowFields` .

Operator	Description
<code>\$sum</code>	Sum of values
<code>\$avg</code>	Average of values
<code>\$min</code>	Minimum value
<code>\$max</code>	Maximum value
<code>\$first</code>	First value in group
<code>\$last</code>	Last value in group
<code>\$push</code>	Array of all values
<code>\$addToSet</code>	Array of unique values
<code>\$stdDevPop</code>	Population standard deviation
<code>\$stdDevSamp</code>	Sample standard deviation
<code>\$count</code>	Count of documents (MongoDB 5.0+)

```
db.sales.aggregate([
  {
    $group: {
      _id: "$region",
      total: { $sum: "$revenue" },
      average: { $avg: "$revenue" },
      highest: { $max: "$revenue" },
      lowest: { $min: "$revenue" },
      firstSale: { $first: "$date" },
      lastSale: { $last: "$date" },
      allSalesmen: { $push: "$salesman" },
      uniqueSalesmen: { $addToSet: "$salesman" }
    }
  }
])
```


6. Array Operators in Aggregation

```
db.orders.aggregate([
  {
    $project: {
      // Size of array
      itemCount: { $size: "$items" },

      // Filter array
      expensiveItems: {
        $filter: {
          input: "$items",
          as: "item",
          cond: { $gt: ["$$item.price", 100] }
        }
      },

      // Map / transform array
      itemNames: {
        $map: {
          input: "$items",
          as: "item",
          in: "$$item.name"
        }
      },

      // Reduce array to single value
      totalCost: {
        $reduce: {
          input: "$items",
          initialValue: 0,
          in: { $add: ["$$value", "$$this.price"] }
        }
      },

      // First/last N elements
      firstTwo: { $slice: ["$items", 2] },
      lastTwo: { $slice: ["$items", -2] },

      // Check if value exists in array
      hasPremium: { $in: ["premium", "$tags"] },
    }
  }
])
```

```
// Array element at index
firstItem: { $arrayElemAt: ["$items", 0] },

// Concatenate arrays
allTags: { $concatArrays: ["$tags", "$categories"] },

// Sort array
sortedPrices: {
  $sortBy: {
    input: "$prices",
    sortBy: { $meta: "textScore" }
  }
}
}
}
}
})
```

7. String Operators

```
db.users.aggregate([
  {
    $project: {
      // Concatenate
      fullName: { $concat: ["$firstName", " ", "$lastName"] },

      // Uppercase / Lowercase
      upperEmail: { $toUpper: "$email" },
      lowerName: { $toLowerCase: "$name" },

      // Substring
      initials: { $substr: ["$name", 0, 1] },

      // String length
      nameLength: { $strLenCP: "$name" },

      // Trim
      cleanName: { $trim: { input: "$name" } },

      // Split
      nameParts: { $split: ["$fullName", " "] },

      // Replace
      cleaned: { $replaceAll: { input: "$bio", find: "bad", replacement: "****" } },

      // Index of substring
      atIndex: { $indexOfCP: ["$email", "@"] },

      // Regex match check
      isGmail: {
        $regexMatch: { input: "$email", regex: /gmail\.com$/ }
      }
    }
  }
])
```


8. Date Operators

```
db.orders.aggregate([
  {
    $project: {
      year:      { $year: "$createdAt" },
      month:     { $month: "$createdAt" },
      day:       { $dayOfMonth: "$createdAt" },
      hour:      { $hour: "$createdAt" },
      minute:    { $minute: "$createdAt" },
      dayOfWeek: { $dayOfWeek: "$createdAt" }, // 1=Sun, 7=Sat
      dayOfYear: { $dayOfYear: "$createdAt" },
      week:      { $week: "$createdAt" },

      // Format date to string
      formattedDate: {
        $dateToString: {
          format: "%Y-%m-%d %H:%M:%S",
          date: "$createdAt"
        }
      },
    },

    // Date difference
    daysOld: {
      $dateDiff: {
        startDate: "$createdAt",
        endDate: "$$NOW",
        unit: "day"
      }
    },

    // Add to date
    expiryDate: {
      $dateAdd: {
        startDate: "$createdAt",
        unit: "month",
        amount: 3
      }
    }
  }
])
```

9. Conditional Operators

\$cond (ternary if-else)

```
db.orders.aggregate([
  {
    $project: {
      status: 1,
      amount: 1,
      // Simple ternary
      label: {
        $cond: {
          if: { $gte: ["$amount", 1000] },
          then: "High Value",
          else: "Standard"
        }
      },
      // Shorthand array syntax
      discount: {
        $cond: [{ $gte: ["$amount", 500] }, 0.1, 0]
      }
    }
  }
])
```

\$switch (multi-branch if-else if)

```
db.products.aggregate([
  {
    $project: {
      priceCategory: {
        $switch: {
          branches: [
            { case: { $lt: ["$price", 50] }, then: "Budget" },
            { case: { $lt: ["$price", 200] }, then: "Mid-Range" },
            { case: { $lt: ["$price", 500] }, then: "Premium" },
            { case: { $gte: ["$price", 500] }, then: "Luxury" }
          ],
          default: "Unknown"
        }
      }
    }
  }
])
```

\$ifNull

```
db.users.aggregate([
  {
    $project: {
      // Use fallback if field is null or missing
      phone: { $ifNull: ["$phone", "Not Provided"] }
    }
  }
])
```

10. \$expr

Allows using **aggregation expressions** inside query operators like `$match`. This enables **comparing two fields** in the same document.

```
// Find orders where discount > 20% of amount
db.orders.aggregate([
  {
    $match: {
      $expr: { $gt: ["$discount", { $multiply: ["$amount", 0.2] }] }
    }
  }
])
```

```
// Find docs where field A > field B (impossible without $expr)
db.inventory.aggregate([
  {
    $match: {
      $expr: { $gt: ["$sold", "$stock"] } // oversold items!
    }
  }
])
```

```
// Combine $expr with logical operators
db.sales.aggregate([
  {
    $match: {
      $expr: {
        $and: [
          { $gte: ["$revenue", 1000] },
          { $lt: ["$cost", "$revenue"] }
        ]
      }
    }
  }
])
```

11. Window Functions (\$setWindowFields)

Available in **MongoDB 5.0+** — similar to SQL window functions (OVER, PARTITION BY).

```

// Running total per category, ranked by date
db.sales.aggregate([
  {
    $setWindowFields: {
      partitionBy: "$category",          // PARTITION BY category
      sortBy: { date: 1 },               // ORDER BY date ASC
      output: {
        // Running cumulative sum
        runningTotal: {
          $sum: "$amount",
          window: { documents: ["unbounded", "current"] }
        },
        // Rank within partition
        rank: { $rank: {} },
        denseRank: { $denseRank: {} },
        rowNumber: { $documentNumber: {} },
        // Moving average (last 3 documents)
        movingAvg: {
          $avg: "$amount",
          window: { documents: [-2, 0] }
        },
        // Lead / Lag
        prevAmount: {
          $shift: { output: "$amount", by: -1, default: 0 }
        },
        nextAmount: {
          $shift: { output: "$amount", by: 1, default: 0 }
        }
      }
    }
  }
])

```

12. ParseError: KaTeX parse error: Expected 'EOF', got '&' at position 7: merge &out

Write aggregation results back to a collection.

```
// $out – replaces the entire target collection
db.orders.aggregate([
  { $group: { _id: "$category", total: { $sum: "$amount" } } },
  { $out: "category_totals" } // creates/replaces collection
])

// $merge – upserts into existing collection (more flexible)
db.orders.aggregate([
  { $group: { _id: "$userId", totalSpent: { $sum: "$amount" } } },
  {
    $merge: {
      into: "user_stats",
      on: "_id", // match key
      whenMatched: "merge", // merge fields if doc exists
      whenNotMatched: "insert" // insert if new
    }
  }
])
```

13. Performance Tips

Best Practices

```
// ❌ BAD – $sort before $match (processes ALL docs)
db.orders.aggregate([
  { $sort: { date: -1 } },
  { $match: { status: "active" } }
])
```

```
// ✅ GOOD – $match first (reduces dataset early)
db.orders.aggregate([
  { $match: { status: "active" } },
  { $sort: { date: -1 } }
])
```

```
// ✅ Use $project early to reduce document size
db.orders.aggregate([
  { $match: { status: "active" } },
  { $project: { amount: 1, userId: 1 } }, // drop unused fields
  { $group: { _id: "$userId", total: { $sum: "$amount" } } }
])
```

✅ Use Indexes

```
// Create index on commonly filtered fields
db.orders.createIndex({ status: 1, date: -1 })

// Check if pipeline uses index
db.orders.explain("executionStats").aggregate([
  { $match: { status: "active" } }
])
```

✅ allowDiskUse for Large Datasets

```
// When pipeline exceeds 100MB memory limit
db.bigCollection.aggregate(
  [ { $group: { _id: "$category", total: { $sum: "$amount" } } } ],
  { allowDiskUse: true }
)
```

Pipeline Stage Order Rule of Thumb

\$match → \$sort → \$skip → \$limit → \$unwind → \$group → \$project → \$lookup

14. Real-World Examples



Sales Dashboard

```
db.orders.aggregate([
  // Step 1: Only completed orders in 2024
  { $match: { status: "completed", year: 2024 } },

  // Step 2: Unwind items array
  { $unwind: "$items" },

  // Step 3: Group by month and category
  {
    $group: {
      _id: { month: "$month", category: "$items.category" },
      revenue: { $sum: { $multiply: ["$items.price", "$items.qty"] } },
      orders: { $sum: 1 }
    }
  },

  // Step 4: Add formatted label
  {
    $addFields: {
      monthLabel: {
        $arrayElemAt: [
          ["", "Jan", "Feb", "Mar", "Apr", "May", "Jun",
            "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"],
          "$_id.month"
        ]
      }
    }
  },

  // Step 5: Sort by month
  { $sort: { "_id.month": 1, revenue: -1 } }
])
```




User Activity Report

```
db.events.aggregate([
  // Filter last 30 days
  {
    $match: {
      timestamp: { $gte: new Date(Date.now() - 30 * 24 * 60 * 60 * 1000) }
    }
  },

  // Join with users collection
  {
    $lookup: {
      from: "users",
      localField: "userId",
      foreignField: "_id",
      as: "user"
    }
  },
  { $unwind: "$user" },

  // Group by user
  {
    $group: {
      _id: "$userId",
      userName: { $first: "$user.name" },
      email: { $first: "$user.email" },
      totalEvents: { $sum: 1 },
      uniqueActions: { $addToSet: "$action" },
      lastActive: { $max: "$timestamp" }
    }
  },

  // Only active users (more than 5 events)
  { $match: { totalEvents: { $gt: 5 } } },

  // Shape output
  {
    $project: {
      _id: 0,
      userId: "$_id",
      userName: 1,
      email: 1,

```

```
totalEvents: 1,
uniqueActionCount: { $size: "$uniqueActions" },
lastActive: 1,
activityLevel: {
  $switch: {
    branches: [
      { case: { $gte: ["$totalEvents", 100] }, then: "Power User" },
      { case: { $gte: ["$totalEvents", 30] }, then: "Active" },
    ],
    default: "Casual"
  }
}
},
{ $sort: { totalEvents: -1 } },
{ $limit: 100 }
])
```



E-Commerce Product Search with Facets

```
db.products.aggregate([
  // Full-text search + price filter
  {
    $match: {
      $text: { $search: "wireless headphones" },
      price: { $gte: 20, $lte: 300 },
      inStock: true
    }
  },

  // Parallel facets for sidebar filters
  {
    $facet: {
      results: [
        { $sort: { score: { $meta: "textScore" }, rating: -1 } },
        { $skip: 0 },
        { $limit: 20 },
        { $project: { name: 1, price: 1, rating: 1, image: 1 } }
      ],
      brandCounts: [
        { $group: { _id: "$brand", count: { $sum: 1 } } },
        { $sort: { count: -1 } }
      ],
      priceBuckets: [
        {
          $bucketAuto: {
            groupBy: "$price",
            buckets: 5,
            output: { count: { $sum: 1 } }
          }
        }
      ],
      totalCount: [
        { $count: "total" }
      ]
    }
  }
])
```



Quick Reference Cheat Sheet

FILTERING	→ \$match (use early, supports indexes)
GROUPING	→ \$group + accumulators (\$sum, \$avg, \$push...)
SHAPING	→ \$project, \$addFields, \$replaceRoot
JOINING	→ \$lookup (left outer join)
ARRAY OPS	→ \$unwind, \$filter, \$map, \$reduce, \$size
CONDITIONALS	→ \$cond, \$switch, \$ifNull
FIELD COMPARE	→ \$expr (compare two fields)
PAGINATION	→ \$sort + \$skip + \$limit
PARALLEL PIPELINES	→ \$facet
RANGE GROUPING	→ \$bucket, \$bucketAuto
WINDOW FUNCTIONS	→ \$setWindowFields (MongoDB 5.0+)
WRITE RESULTS	→ \$out, \$merge

Guide covers MongoDB 4.4 – 7.x | All examples tested with MongoDB Community Edition